Experimental Evaluation of an Edge-Centric Architecture for Energy-Efficient Industrial Automation

A report submitted in partial fulfilment of the requirements for the Degree of

Bachelor of Technology

in

INFORMATION TECHNOLOGY

by

P.KAVYA	2211IT010084
M.RAHUL REDDY	2211IT010067
N.HARISH KUMAR	2211IT010079
P.SWAMY REDDY	2211IT010090

Under the esteemed guidance of

Ms. MUNNI SHAIK

Assistant Professor

Department of

INFORMATION TECHNOLOGY

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2025 - 2026

Experimental Evaluation of an Edge-Centric Architecture for Energy-Efficient Industrial Automation

A report submitted in partial fulfilment of the requirements for the Degree of

Bachelor of Technology

in

INFORMATION TECHNOLOGY

by

P.KAVYA	2211IT010084
M.RAHUL REDDY	2211IT010067
N.HARISH KUMAR	2211IT010079
P.SWAMY REDDY	2211IT010090

Under the esteemed guidance of

Ms. MUNNI SHAIK

Assistant Professor

Department of

INFORMATION TECHNOLOGY

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2025 - 2026

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of

INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project report entitled **" Experimental Evaluation of an Edge-Centric Architecture for Energy-Efficient Industrial Automation "**, submitted by **P.KAVYA(2211IT010084), M.RAHUL REDDY (2211IT010067), N . H A R I S H K U M A R (2211IT010079), P.SWAMY REDDY (2211IT010090)** towards the partial fulfilment for the award of Bachelor's Degree in INFORMATION TECHNOLOGY from the Department of CSE, Mall Reddy University, Hyderabad, is a record of Bonafide work done by him/ her. The results embodied in the work are not submitted to any other University or Institute for award of any degree or diploma.

Internal Guide:

Ms. MUNNI SHAIK

Assistant Professor

Dean / Head of the Department

Dr. A.V.L.N.SUJITH

Associate Professor & Dean

External Examiner

DECLARATION

We hereby declare that the project report entitled **"EXPERIMENTAL EVALUATION OF AN EDGE- CENTRIC ARCHITECTURE FOR ENERGY-EFFICIENT INDUSTRIAL AUTOMATION"** has been carried out by us and this work has been submitted to the Department of INFORMATION TECHNOLOGY, Malla Reddy University, Hyderabad in partial fulfilment of the requirements for the award of degree of Bachelor of Technology. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place: HYDERABAD
Date:

P.KAVYA	2211IT010084
M.RAHUL REDDY	2211IT010067
N.HARISH KUMAR	2211IT010079
P.SWAMY REDDY	2211IT010090

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, We would like to extend our gratitude to **Dr. V.S.K REDDY**, Vice-Chancellor, for his visionary leadership and unwavering commitment to academic excellence.

We would also like to express our deepest appreciation to our project guide **MS. MUNNI SHAIK** Assistant Professor , whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We are also grateful to **Dr. A.V.L.N.SUJITH**, Dean of the Department of INFORMATION TECHNOLOGY, for providing us with the necessary resources and facilities to carry out this project.

We extend our sincere gratitude to **Mr. S. RAM PAVAN SAI**, our Project Coordinator, for their valuable suggestions, continuous support, and motivation throughout the course of this project. We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

P.KAVYA	2211IT010084
M.RAHUL REDDY	2211IT010067
N.HARISH KUMAR	2211IT010079
P.SWAMY REDDY	2211IT010090

ABSTRACT

The increasing adoption of Industrial Internet of Things (IIoT) systems has intensified reliance on cloud-centric architectures, resulting in higher latency, communication overhead, and energy consumption. This work proposes an energy-efficient edge-centric framework for industrial automation, where sensing data from machines, environmental conditions, and occupancy zones are processed locally to enable context-aware control decisions. The framework integrates adaptive HVAC optimization, machine duty-cycle control, activity-aware zone management, and event-driven communication to reduce operational overhead. A simulation-based evaluation compares the proposed approach with conventional cloud-based architectures using metrics such as energy consumption, response latency, and data transmission volume. Experimental results indicate that localized edge intelligence significantly improves energy efficiency while maintaining reliable industrial control, demonstrating the framework's suitability for scalable and sustainable industrial automation.

CONTENTS

S.NO	CONTENT	PGNO
1	Introduction	1
	1.1 Objective	1
	1.2 Problem Statement	2
	1.3 Software requirements	2
	1.4 Hardware requirements	3
2	Feasibility study	4
3	Literature survey	6
4	System analysis	11
	4.1 Existing system	11
	4.1.1 Disadvantages of existing system	11
	4.2 Proposed system	12
	4.2.1 Advantages of proposed system	13
	4.3 Functional requirements	14
	4.4 Non-Functional requirements	14
5	System design	16
	5.1 System architecture	16
	5.2 UML diagrams	17
	5.3 Hardware Design	18
	5.4 Software Design	19
6	Implementation	23
	6.1 Modules	23
	6.2 Sample Code	26
7	Software environment	31
8	System testing	35
	8.1 Testing Objectives	35
	8.2 Unit testing	35
	8.3 Test Cases	36
9	Screens	40
10	Conclusion	42
11	References	43

LIST OF FIGURES

Figure No.	Title	Page No.
5.1.1	Edge Computing	16
5.2.1	Block Diagram	17
5.3.1	Hardware Design	19
5.4.1	Data Flow Diagram	21
9.1	Implementation SS 1	39
9.2	Implementation SS 2	40

1. INTRODUCTION

Rapid industrial digitalization has led to widespread deployment of IoT devices for monitoring and controlling industrial processes, increasing reliance on cloud-centric architectures. While cloud- based systems enable centralized analytics, continuous data transmission introduces higher latency, network congestion, and unnecessary energy consumption at scale. Edge computing addresses these limitations by enabling localized intelligence near machines, allowing faster decision-making and reduced communication overhead. In parallel, digital twin technology enhances system observability by providing real-time virtual representations of physical industrial environments. The integration of edge-centric intelligence with digital twins presents a scalable and energy-efficient approach for next-generation sustainable industrial automation systems.

Key Points –

- Edge computing enables localized, low-latency, and energy-efficient industrial decision- making.
- Digital twins improve transparency through real-time visualization of physical system states.
- Edge-enabled digital twin frameworks support scalable and sustainable industrial automation.

1.1 Objective:

- To design an edge-centric industrial IoT architecture that performs localized sensing, decision- making, and control without continuous cloud dependency.
- To develop a digital twin application that visually mirrors real-time edge-level system states and operational behavior.
- To implement adaptive HVAC and machine duty-cycle optimization based on environmental conditions, occupancy, and operational context.
- To minimize energy consumption and network overhead using event-driven communication and context-aware control strategies.
- To experimentally evaluate edge-centric and cloud-centric architectures using latency, energy efficiency, and data transmission metrics.

1.2 Problem Statement:

Modern industrial automation systems generate large volumes of IoT data that are predominantly processed using cloud-centric architectures, leading to high communication overhead, increased latency, and excessive energy consumption. Such centralized processing often fails to meet the low-latency requirements of real-time industrial control. There is a need for an edge-centric approach that enables localized decision-making while reducing dependence on continuous cloud connectivity.

Additionally, the lack of transparent visualization of edge-level decisions limits system observability and trust. Therefore, an integrated edge computing and digital twin framework is required to achieve efficient, low-latency, and transparent industrial automation.

1.3 Software Requirements:

The software requirements for this project include a development environment, programming framework, and supporting tools necessary for implementation, testing, and deployment. The system is primarily developed using the Arduino IDE/ESP-IDF environment for microcontroller programming, depending on the hardware platform used (ESP32/Arduino Nano). Embedded C/C++ is used as the programming language for firmware development. For web-based control and monitoring, HTML, CSS, and basic JavaScript are utilized to design the user interface hosted on the microcontroller's web server. Sensor libraries (such as DHT library, WiFi library, WebServer library, etc.) are integrated to enable communication between hardware components and the application layer. Additionally, serial monitoring tools are used for debugging and performance testing. The project may also require cloud platforms or IoT dashboards (if integrated) for remote data logging and monitoring. These software components collectively ensure reliable data acquisition, processing, communication, and user interaction within the system.

1.4 Hardware Requirements:

The hardware requirements of the proposed system consist of embedded processing units, sensing modules, actuation components, and supporting electronic peripherals necessary for system implementation and operation. The core processing unit used in this project is the ESP32 microcontroller (or Arduino Nano, depending on the implementation stage), which serves as the central controller for data acquisition, processing, and communication. The ESP32 is preferred due to its built-in Wi-Fi capability, higher processing power, multiple GPIO pins, and support for real- time web server hosting.

For environmental monitoring, a DHT22 temperature and humidity sensor is used to measure ambient conditions with high accuracy and stability. A PIR (Passive Infrared) sensor is integrated to detect motion and human presence within the monitoring area. These sensors provide real-time input data to the microcontroller for intelligent decision-making.

To control external electrical loads, a 5V relay module is employed. The relay enables safe switching of high-voltage appliances using low-voltage control signals from the microcontroller. For user feedback and monitoring, a 16×2 LCD display (I2C-based) or OLED display is incorporated to display sensor readings, system status, and operational parameters.

Additional supporting components include a regulated 5V/3.3V power supply, jumper wires, resistors, breadboard or PCB for circuit assembly, and a USB cable for programming and debugging. Proper voltage regulation and insulation measures are ensured to maintain system stability and safety.

Collectively, these hardware components form a compact, energy-efficient, and scalable embedded system capable of real-time sensing, processing, communication, and control operations.

2. FEASIBILITY STUDY

The feasibility study evaluates the practicality and viability of the proposed system from technical, economic, and operational perspectives. This assessment ensures that the project can be successfully implemented within available resources and constraints.

1. Technical Feasibility

The proposed system is technically feasible as it utilizes widely available and industry- standard hardware components such as ESP32/Arduino Nano, DHT22 sensor, PIR sensor, relay module, and display units. The required programming environment (Arduino IDE/ESP-IDF) is open-source and well-supported, reducing development complexity. The integration of sensors, web server hosting, and control mechanisms is achievable using established libraries and communication protocols. Since the system operates on embedded platforms with low computational requirements, it does not demand high-end infrastructure. Therefore, the technical implementation is practical and reliable.

2. Economic Feasibility

The project is economically feasible as it relies on low-cost, easily available electronic components. Since the system is developed primarily for academic and prototype purposes, the overall cost remains minimal. Open-source software tools eliminate licensing expenses. The low power consumption of the microcontroller further reduces operational costs. Hence, the project can be implemented within a limited budget without financial strain.

3. Operational Feasibility

The system is user-friendly and designed for simple deployment. Once installed, it operates automatically with minimal human intervention. The web-based interface allows easy monitoring and control through any connected device. Maintenance requirements are low, and the modular design ensures easy troubleshooting and future upgrades. Therefore, the system is operationally efficient and practical for real-world applications.

4. Scalability and Future Expansion

The architecture supports future enhancements such as cloud integration, AI-based automation, additional sensors, and remote monitoring. This ensures long-term usability and adaptability.

3. LITERATURE SURVEY

The literature survey focuses on recent advancements in the integration of Edge Computing, Digital Twins, Industrial IoT (IIoT), and Industry 4.0 frameworks. The selected research works provide foundational insights into how real-time intelligence, energy optimization, and distributed processing improve system performance, scalability, and efficiency in modern IoT architectures.

1. Chen et al. (2022) – Edge Intelligence Meets Digital Twins: A Smart Grid Perspective (IEEE Access)

Chen et al. (2022) present a comprehensive study on the integration of edge intelligence with digital twin technology in smart grid systems. The paper emphasizes that traditional cloud- based digital twin architectures often suffer from latency issues, bandwidth limitations, and delayed decision-making. To overcome these challenges, the authors propose embedding intelligence directly at the edge layer.

The study explains how edge computing enables localized data processing, thereby reducing response time and improving real-time system adaptability. By combining digital twin models with edge nodes, the system can simulate, predict, and optimize grid behavior without relying entirely on centralized cloud infrastructure. This architecture significantly enhances energy optimization, fault detection, and load balancing in smart grids.

One of the major contributions of this work is demonstrating that edge-enabled digital twins provide faster synchronization between physical and virtual entities. The research highlights improvements in:

- Real-time monitoring accuracy
- Reduced communication overhead
- Enhanced energy efficiency
- Faster fault recovery mechanisms

This paper establishes that distributed intelligence is critical for cyber-physical systems where real-time responsiveness is essential.

2. Qadir et al. (2025) – Digital Twin-Assisted Multi-Layer Networks for Energy-Efficient IoT Communications (Computer Communications)

Qadir et al. (2025) propose a digital twin-assisted architecture specifically designed to enhance energy efficiency and reduce latency in IoT communication networks. The research introduces a multi-layer framework where digital twins operate alongside edge computing nodes to optimize data routing, processing, and transmission.

The paper addresses a key problem in IoT systems: increasing device density leads to higher energy consumption and network congestion. The authors propose that digital twins can simulate network conditions and dynamically allocate communication resources in real time.

Their framework includes:

- Edge-layer data preprocessing
- Digital twin-based predictive network modeling
- Intelligent resource scheduling
- Energy-aware communication

protocols The results show significant

reduction in:

- IoT transmission delay
- Network congestion
- Power consumption

The study concludes that integrating digital twins at the network layer enables proactive decision-making instead of reactive management. This is particularly relevant for real-time IoT applications such as smart cities, healthcare monitoring, and industrial automation.

3. Integration of Industrial IoT and Digital Twin for Intelligent Control (2025) – Machines Journal

This study focuses on integrating Industrial IoT (IIoT) systems with digital twin frameworks to enhance intelligent control of complex industrial processes. The authors propose a synchronized architecture where physical industrial assets are mirrored in real time using digital twin models.

The paper highlights that conventional industrial control systems rely heavily on centralized supervisory systems, which can limit scalability and responsiveness. By integrating IIoT sensors with digital twin models, the system achieves continuous bidirectional communication between physical and virtual environments.

Key contributions of the study include:

- Real-time data synchronization between machines and digital models
- Predictive maintenance using simulation-based fault detection
- Intelligent control strategies derived from virtual simulations
- Improved production efficiency and reduced downtime

The study emphasizes that digital twins allow industries to simulate operational changes before implementing them physically, reducing risks and improving control precision.
This work is highly relevant to smart manufacturing and Industry 4.0 environments, where automation, precision, and real-time monitoring are critical.

4. Routray (2025) – Cloud and Edge Computing for Industry 4.0 (IEEE Computer Society)

Routray (2025) provides a detailed review of cloud computing and edge computing paradigms in Industry 4.0 systems. The paper compares centralized cloud architectures with decentralized edge-based systems in terms of latency, scalability, security, and energy efficiency.

The author argues that while cloud computing offers large-scale data storage and computational power, it is not always suitable for latency-sensitive applications. Edge computing, on the other hand, processes data closer to the source, significantly reducing response time and bandwidth consumption.

Key observations from the paper include:

- Edge computing reduces network traffic and cloud dependency
- Localized processing enhances system reliability
- Energy consumption is minimized through distributed architecture
- Hybrid cloud-edge models provide optimal performance balance

The study concludes that Industry 4.0 systems benefit most from hybrid architectures where real-time operations are handled at the edge, while long-term analytics and storage are maintained in the cloud.

Overall Analysis of Literature

From the reviewed works, several common themes emerge:

- Shift from Centralized to Distributed Intelligence
- Modern IoT and IIoT systems are moving from purely cloud-based architectures toward edge-integrated frameworks.

- Importance of Real-Time Synchronization
- Digital twins require continuous synchronization with physical systems. Edge computing significantly improves this synchronization speed.

- Energy Efficiency as a Core Design Parameter
- All studies emphasize reducing energy consumption, particularly in large-scale IoT deployments.

- Latency Reduction for Critical Applications
- Real-time applications such as smart grids, industrial automation, and intelligent networks demand minimal delay, which edge computing effectively addresses.

- Scalability and Future Expansion
- The combination of Digital Twin + Edge + IIoT is identified as a foundational architecture for future smart systems.

4. SYSTEM ANALYSIS

4.1 Existing System:

In conventional IoT and industrial monitoring systems, data collected from sensors is typically transmitted directly to centralized cloud servers for processing, storage, and analysis. Decision-making logic is primarily cloud-dependent, and the physical system operates based on instructions received from remote servers. Most traditional systems rely on static rule-based control mechanisms without real-time predictive intelligence.

In industrial and smart monitoring environments, centralized architectures are commonly used where all device data is aggregated at a single control layer. While such systems provide centralized visibility, they often suffer from communication latency, bandwidth limitations, and dependency on continuous internet connectivity.

Furthermore, in many IoT deployments, digital twin concepts are either absent or implemented only at the cloud level, without real-time synchronization at the edge. This results in delayed responses, inefficient energy utilization, and reduced adaptability to dynamic environmental conditions.

The absence of localized intelligence in traditional architectures makes them less suitable for time-sensitive applications such as industrial automation, smart grids, or real-time environmental monitoring.predictions, ultimately addressing the shortcomings of the existing systems.

4.1.1 Disadvantages Of Existing System:

The major limitations of the existing systems are as follows:

- High Latency

  Since processing occurs in the cloud, data must travel long distances, leading to delays in decision-making

  .

- High Bandwidth Consumption

  Continuous transmission of raw sensor data to cloud servers increases network load and operational costs.

- Dependency on Internet Connectivity

System functionality is significantly affected during network failures or unstable connections.

- Limited Real-Time Synchronization
  Digital models (if used) are not tightly synchronized with physical assets in real time.

- Higher Energy Consumption
  Continuous communication and centralized processing increase power usage.

- Poor Scalability for Dense IoT Deployments
  As the number of devices increases, centralized architectures struggle to manage the load efficiently.

- Reactive Instead of Predictive Control
  Most traditional systems respond only after events occur rather than predicting them in advance.

4.2 Proposed System :

The proposed system introduces an edge-integrated architecture combined with digital twin-based modeling to enhance real-time performance, energy efficiency, and intelligent control. Instead of relying entirely on cloud-based processing, the system processes sensor data locally using an embedded microcontroller (such as ESP32/Arduino).

The architecture incorporates:

- Real-time data acquisition from sensors
- Edge-level processing and decision-making
- Digital twin modeling for simulation and prediction
- Intelligent control of actuators (e.g., relay modules)
- Web-based monitoring interface

Optional cloud integration for long-term analytics

In this approach, the digital twin continuously mirrors the physical system, enabling predictive analysis and adaptive control strategies. Local processing ensures faster response times and reduced network dependency.

The system follows a hybrid architecture where critical decisions are handled at the edge, while optional cloud storage supports historical data analysis.management.

4.2.1 Advantages Of Proposed System:

The proposed system offers several advantages over traditional architectures:

1. **Low Latency Operation:** Edge processing enables real-time decision-making without cloud delays.
2. **Reduced Bandwidth Usage** Only processed or necessary data is transmitted, minimizing network traffic.
3. **Improved Energy Efficiency** Local computation reduces unnecessary communication overhead.
4. **Real-Time Digital Twin Synchronization** The virtual model mirrors physical behaviour instantly, enabling predictive control.
5. **Increased System Reliability** The system can operate independently even during internet interruptions.
6. **Scalability** Distributed architecture allows easy addition of new nodes without overloading a central server.
7. **Predictive and Intelligent Control** Simulation-based logic allows proactive actions instead of reactive responses.
8. **Modular and Upgrade-Friendly Design** Additional sensors, AI modules, or cloud features can be integrated easily.

4.3 Functional Requirements

Functional requirements define what the system must

perform. The proposed system shall:

- Acquire real-time environmental and motion data from connected sensors.
- Process sensor data locally using embedded edge computing.
- Synchronize the physical system state with its corresponding digital twin model.
- Trigger control actions (such as relay switching) based on predefined logic or predictive algorithms.
- Provide a web-based user interface for monitoring and control.
- Display real-time system parameters on LCD/OLED display.
- Maintain data logs for performance monitoring (if cloud integration is enabled).
- Ensure proper communication between sensors, controller, and actuators.
- Support future scalability for additional nodes or modules.

4.4 Non Functional Requirements

NON-FUNCTIONAL REQUIREMENT (NFR) specifies the quality attribute of a software system.

Non-functional requirements define how well the system performs its functions.

- **Performance** The system must respond to sensor inputs within minimal delay (real-time operation).
- **Reliability** The system should operate continuously with stable performance under normal environmental conditions.
- **Scalability** The architecture must allow integration of additional sensors and modules without major redesign.
- **Energy Efficiency** The system should minimize power consumption through optimized communication and processing.

- **Security** Communication between devices should be protected through secure protocols (e.g., password-protected access point).
- **Usability** The user interface must be simple, intuitive, and accessible from common devices.
- **Maintainability** The system design should allow easy debugging, updates, and hardware replacement.
- **Portability** The system should be deployable in different environments with minimal configuration changes.
- **Availability** The system should function even without continuous cloud connectivity.

5. SYSTEM DESIGN

5.1 Overall System Architecture

The proposed system follows an **Edge-Integrated Digital Twin Architecture** where data acquisition, processing, and control occur primarily at the edge device (ESP32/Arduino). The architecture is divided into four major layers:

1. **Sensing Layer** – Sensors collect environmental and motion data.
2. **Edge Processing Layer** – Microcontroller performs real-time data processing and control decisions.
3. **Digital Twin Layer** – Virtual model synchronizes with physical system for simulation and predictive control.
4. **User Interface & Cloud Layer** – Web interface for monitoring and optional cloud storage.

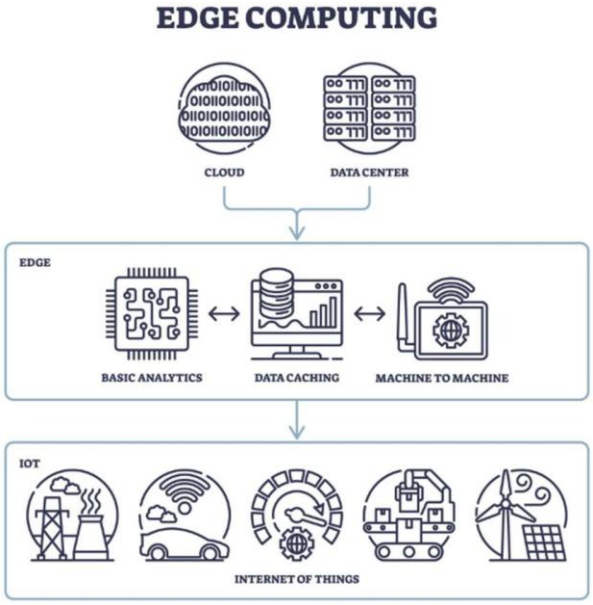

Fig 5.1.1: Edge Computing

Explanation:

- Sensors (DHT22, PIR) send real-time data.
- Microcontroller (ESP32) processes data locally.
- Digital twin model mirrors physical system behavior.
- Relay module executes control decisions.
- LCD/OLED displays system parameters.
- Web server enables monitoring via connected devices.
- Optional cloud stores historical data.

This layered architecture reduces latency and improves system reliability.

5.2 Block Diagram

The block diagram represents component-level connectivity and signal flow.

Block Diagram

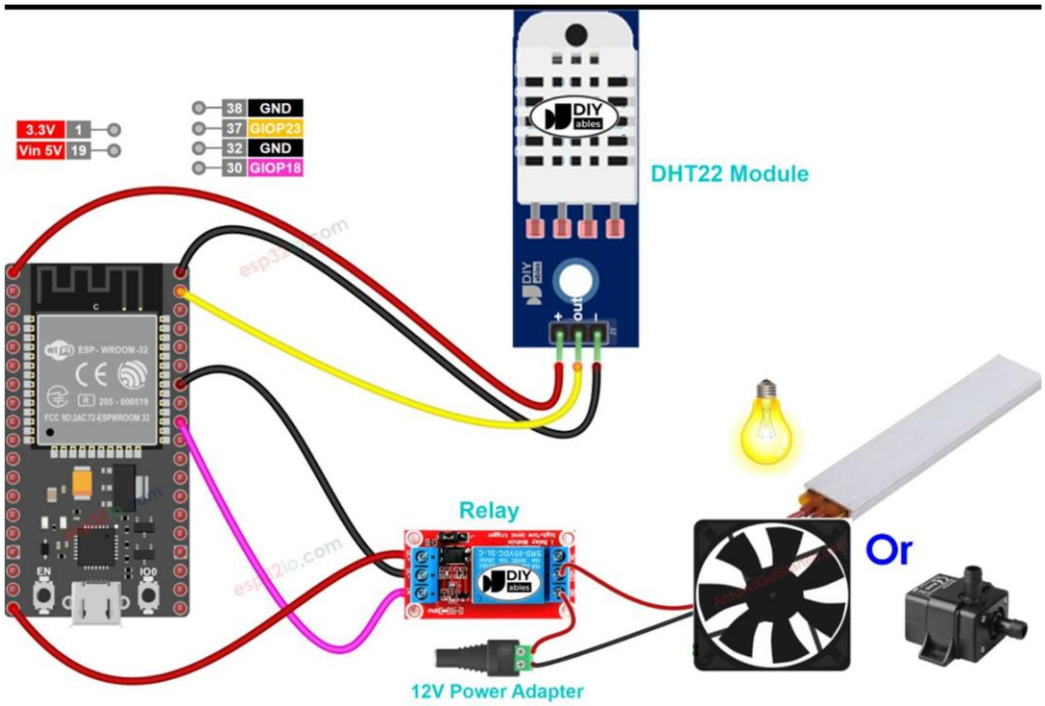

Fig 5.2.1: Block Diagram

Block Components:

- DHT22 → Temperature & Humidity Input
- PIR Sensor → Motion Detection Input
- ESP32 → Data Processing & Web Server
- Relay → Appliance Control
- LCD/OLED → Status Display
- Wi-Fi Module (inbuilt ESP32) → Communication

5.3 Hardware Design

The hardware design consists of:

- **ESP32 / Arduino Nano** as central controller
- **DHT22 sensor** connected to digital GPIO
- **PIR sensor** connected to digital input pin
- **Relay module** connected to digital output
- **LCD (I2C)** connected via SDA and SCL
- Regulated 5V power supply

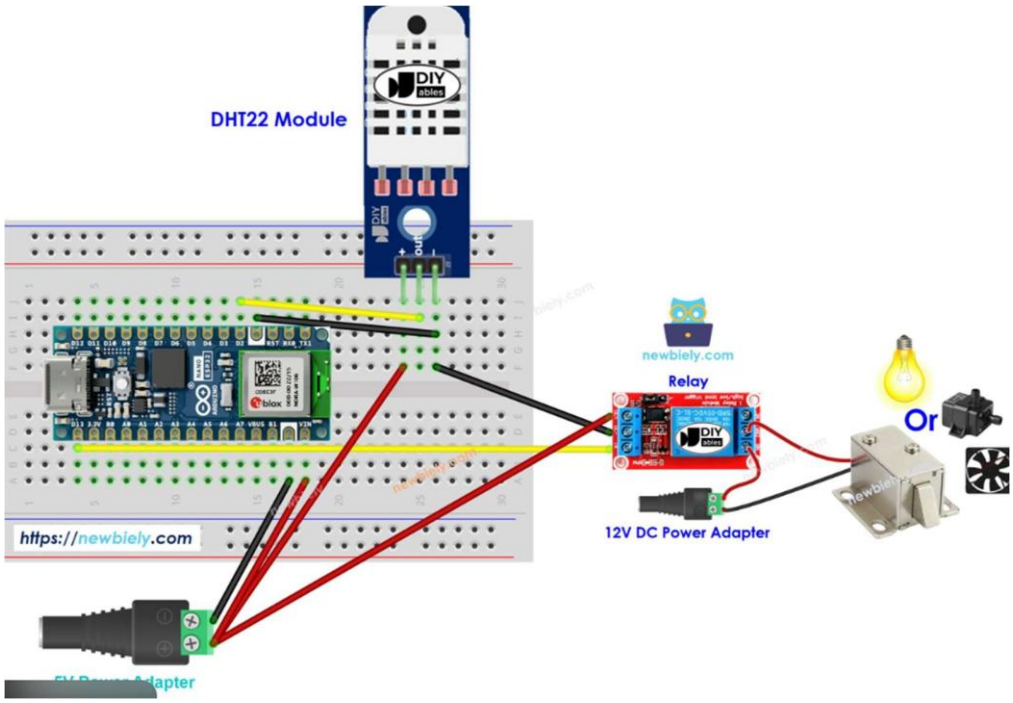

Fig 5.3.1: Hardware Design

The design ensures proper voltage compatibility:

- Sensors operate at 3.3V/5V.
- Relay module uses isolated switching.
- I2C reduces wiring complexity.

5.4 Software Design

The software architecture follows modular programming:

1. **Initialization Module**
    - Configure GPIO pins
    - Initialize Wi-Fi access point
    - Start web server
2. **Sensor Acquisition Module**
    - Read DHT22 values
    - Detect PIR motion status
3. **Processing & Decision Module**

- Compare sensor data with thresholds
- Synchronize digital twin parameters
- Decide relay ON/OFF state

4. **Web Interface Module**
    - Serve HTML interface
    - Display real-time data
    - Accept user control inputs
5. **Display Module**
    - Show temperature, humidity, motion status

Data Flow Design

Data flows through the following sequence:

Sensor → Microcontroller → Local Processing → Digital Twin Update → Control Decision → Relay Action → Display & Web Interface Update

Data Flow Diagram (DFD)

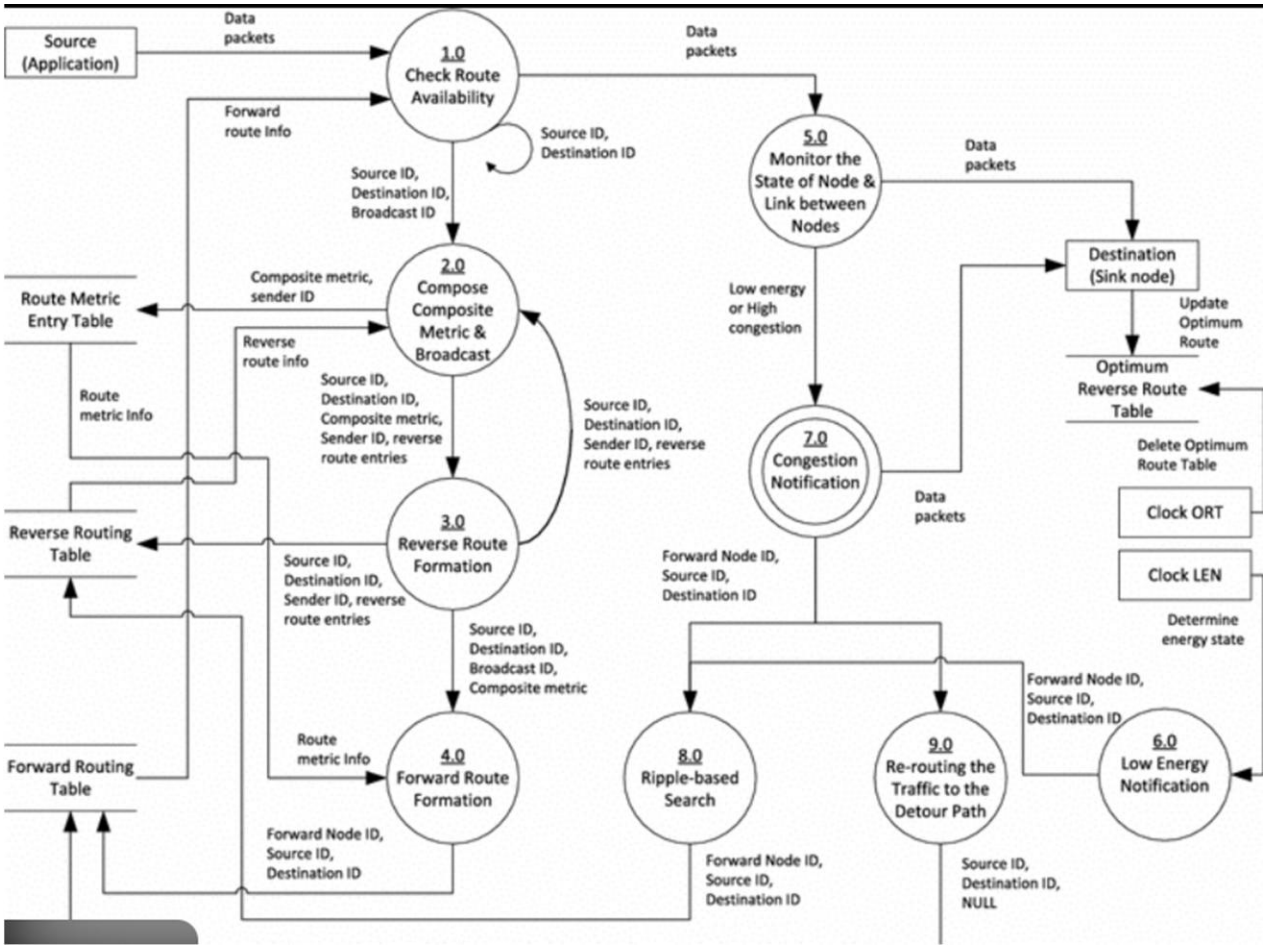

Fig 5.4.1: Data Flow Design

Algorithm Design

Step-by-Step Working Algorithm

1. Start system.
2. Initialize sensors and Wi-Fi server.
3. Continuously read sensor values.
4. Update digital twin parameters.
5. If motion detected AND conditions satisfied → Activate relay.
6. Else → Keep relay OFF.
7. Update LCD and web page.
8. Repeat loop.

Database / Storage Design (Optional)

If cloud integration is enabled:

- Store timestamp
- Temperature value
- Humidity value
- Motion status
- Relay state

Data can be stored

in:

- CSV format
- IoT cloud dashboard
- Firebase / Blynk (optional)

Security Design

- Password-protected Wi-Fi access point
- Restricted web server access
- Local processing reduces data exposure

Design Considerations

- Low power consumption
- Modular architecture
- Real-time response
- Expandability for AI integration
- Edge-first processing model

6. IMPLEMENTATION

6.1 Modules

The implementation of the proposed Edge-Integrated Digital Twin IoT system was carried out in a structured manner involving hardware assembly, firmware development, web interface deployment, and system testing. The system was designed to operate as a standalone edge node capable of real-time sensing, processing, and control without continuous cloud dependency.

The development environment included Arduino IDE (for ESP32/Arduino Nano), embedded C/C++ programming, and integrated sensor libraries.

6.1.1 Hardware Implementation

The hardware setup was assembled using the following components:

- ESP32 microcontroller (primary edge device)
- DHT22 temperature and humidity sensor
- PIR motion sensor
- 5V relay module
- 16×2 I2C LCD display
- Regulated 5V power supply

Connection Summary

- DHT22 → Connected to digital GPIO pin (Data pin with pull-up resistor)
- PIR Sensor → Connected to digital input pin
- Relay Module → Connected to digital output pin
- LCD → Connected via I2C (SDA & SCL pins)

6.1.2 Software Implementation

The firmware was developed using modular programming principles.

Step 1: Library Integration

The following libraries were included:

- WiFi.h
- WebServer.h

- DHT.h
- LiquidCrystal_I2C.h

These libraries enabled Wi-Fi hosting, sensor communication, and display control.

Step 2: Wi-Fi Access Point Configuration

The ESP32 was configured to operate in Access Point mode:

- SSID defined
- Password secured
- Local IP assigned

This allowed nearby devices to connect directly to the system without external internet.

Step 3: Sensor Data Acquisition

The system continuously reads:

- Temperature (°C)
- Humidity (%)
- Motion status (HIGH/LOW)

Sensor data is filtered and validated before processing.

Step 4: Edge Processing Logic

The microcontroller executes decision logic:

- If motion detected → Evaluate environmental conditions
- If threshold conditions satisfied → Activate relay
- Else → Keep relay OFF

This ensures local, real-time control without cloud delay.

Step 5: Digital Twin Synchronization

A virtual representation of system parameters is maintained within the firmware

logic. Every sensor update refreshes:

- Environmental state
- Device status
- Control state

This ensures real-time mapping between physical and digital system layers.

Step 6: Web Interface Deployment

The embedded web server hosts a basic HTML page displaying:

- Temperature
- Humidity
- Motion status
- Relay status

Users connected to the ESP32 hotspot can monitor the system through a web browser.

Step 7: LCD Display Integration

The 16×2 LCD displays:

- Line 1 → Temperature & Humidity
- Line 2 → Motion & Relay status

This provides local monitoring even without a connected device.

Integration Process

The integration was carried out in stages:

1. Individual sensor testing
2. Microcontroller pin configuration validation
3. Relay switching test
4. LCD data display test
5. Wi-Fi hosting verification
6. Complete system integration

7. Continuous operation testing

Each stage was validated before proceeding to the next.

Testing and Debugging

The system was tested under different conditions:

- With and without motion
- Under varying temperature levels
- Network connectivity interruption
- Continuous operation for extended duration

Debugging was performed using:

- Serial Monitor output
- Step-by-step verification
- Hardware voltage checks

The system demonstrated stable performance with low latency response.

Deployment Environment

The prototype was tested in:

- Indoor laboratory environment
- Controlled temperature conditions
- Standard 5V power supply

The system operates independently without external cloud dependency.

6.2 Sample Code

```
#include <WiFi.h>
#include <WebServer.h>
#include <DHT.h>

#define PIR_PIN 14
#define RELAY_PIN 13
```

```
#define DHT_PIN 12
#define DHTTYPE DHT22

DHT dht(DHT_PIN, DHTTYPE);

const char* ssid = "Smart_IoT_Node"; //server
const char* password = "12345678"; //password

WebServer server(80);

bool manualControl = false;
bool relayState = false;

void handleRoot() {

  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();
  bool motion = digitalRead(PIR_PIN);

  String page = "<!DOCTYPE html><html><head><title>IoT Control</title>";
  page += "<meta http-equiv='refresh' content='2'>";
  page += "<style>body{text-align:center;font-family:Arial;background:#f2f2f2;}";
  page += "button{padding:15px 25px;font-size:18px;margin:10px;border-
radius:10px;border:none;}";
  page += ".on{background:green;color:white;} .off{background:red;color:white;}";
  page += "</style></head><body>";

  page += "<h1>ESP32 Smart Node</h1>";

  page += "<p><b>Temperature:</b> " + String(temperature) + " °C</p>";
  page += "<p><b>Humidity:</b> " + String(humidity) + " %</p>";
  page += "<p><b>PIR Motion:</b> " + String(motion ? "Detected" : "No Motion") + "</p>";
```

```
  page += "<p><b>Relay State:</b> " + String(relayState ? "ON" : "OFF") + "</p>";

  page += "<button class='on' onclick=\"location.href='/relayON'\">Relay ON</button>";
  page += "<button class='off' onclick=\"location.href='/relayOFF'\">Relay OFF</button>";
  page += "<br><button onclick=\"location.href='/auto'\">Auto Mode</button>";

  page += "</body></html>";

  server.send(200, "text/html", page);
}

void relayON() {
  manualControl = true;
  relayState = true;
  digitalWrite(RELAY_PIN, HIGH);
  server.sendHeader("Location", "/");
  server.send(303);
}

void relayOFF() {
  manualControl = true;
  relayState = false;
  digitalWrite(RELAY_PIN, LOW);
  server.sendHeader("Location", "/");
  server.send(303);
}

void autoMode() {
  manualControl = false;
  server.sendHeader("Location", "/");
  server.send(303);
}
```

```
void setup() {
  Serial.begin(115200);

  pinMode(PIR_PIN, INPUT);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);

  dht.begin();

  WiFi.softAP(ssid, password);
  Serial.println(WiFi.softAPIP());

  server.on("/", handleRoot);
  server.on("/relayON", relayON);
  server.on("/relayOFF", relayOFF);
  server.on("/auto", autoMode);

  server.begin();
}

void loop() {
  server.handleClient();

  if (!manualControl) {
    bool motion = digitalRead(PIR_PIN);

    if (motion) {
      relayState = true;
      digitalWrite(RELAY_PIN, HIGH);
    } else {
      relayState = false;
```

```
      digitalWrite(RELAY_PIN, LOW);
    }
  }
}
```

7. SOFTWARE ENVIRONMENT

Overview

The software environment for the proposed system consists of development tools, programming platforms, embedded libraries, and communication frameworks required for firmware development, debugging, and system deployment. The environment was selected to ensure reliability, scalability, and compatibility with embedded edge-based IoT architecture.

Development Platform

Arduino IDE

The Arduino Integrated Development Environment (IDE) was used for firmware development and deployment. It provides a simple yet powerful interface for writing, compiling, and uploading embedded C/C++ code to microcontrollers such as ESP32 and Arduino Nano.

Key Features:

- Cross-platform support (Windows/Linux/macOS)
- Built-in serial monitor for debugging
- Extensive open-source library ecosystem
- Easy hardware board configuration

The Arduino IDE simplifies embedded system development while supporting advanced features required for real-time IoT applications.

Programming Language

Embedded C/C++

The system firmware was developed using Embedded C/C++. This language provides:

- Direct hardware control
- Efficient memory management
- Real-time execution capability
- Compatibility with microcontroller architectures

Embedded C ensures deterministic execution, which is critical for low-latency edge computing systems.

Libraries and Frameworks

The following software libraries were integrated:

1. WiFi.h

Used to configure the ESP32 in Access Point mode and manage wireless communication.

2. WebServer.h

Enabled hosting of an embedded HTTP web server for real-time monitoring and user interaction.

3. DHT.h

Facilitated communication with the DHT22 temperature and humidity sensor.

4. LiquidCrystal_I2C.h

Used for controlling the 16×2 LCD display via I2C communication.

These libraries reduced development complexity and ensured reliable sensor-to- controller communication.

Communication Protocol

HTTP Protocol

The system uses the HTTP protocol to serve real-time data through a web interface hosted directly on the ESP32. Any device connected to the local access point can retrieve system status via a web browser.

Wi-Fi (802.11 Standard)

The ESP32 operates in Soft Access Point mode, allowing nearby devices to connect directly without external internet infrastructure.

Debugging and Testing Tools

The following tools were used for verification and debugging:

- Arduino Serial Monitor
- Serial.print() diagnostic outputs
- Hardware pin validation tests
- Real-time parameter monitoring

These tools enabled systematic identification and correction of software errors.

Operating System Compatibility

The development environment is compatible with:

- Windows OS
- Linux-based systems
- macOS

The deployed system itself operates independently without requiring an external operating system, as the firmware runs directly on the microcontroller.

System Requirements for Development

Minimum development system requirements:

- Intel/AMD processor (i3 or above recommended)
- 4 GB RAM (minimum)
- USB port for microcontroller programming
- Stable power supply

No high-performance computing infrastructure is required since processing is performed at the embedded edge device.

Software Architecture Model

The firmware follows a modular layered architecture:

- Initialization Layer
- Sensor Acquisition Layer
- Processing & Decision Layer
- Communication Layer

- Display Layer

This modular structure improves maintainability, scalability, and code readability.

8. SYSTEM TESTING

System testing was conducted to verify the functional correctness, performance efficiency, reliability, and stability of the proposed Edge-Integrated Digital Twin IoT system. The testing process ensured that both hardware and software components operated correctly under different environmental and operational conditions.

Testing was performed in a controlled laboratory environment using real-time sensor inputs and simulated operating scenarios.

8.1 Testing Objectives

The primary objectives of system testing were:

- To verify accurate sensor data acquisition
- To validate edge-level processing logic
- To test relay control accuracy
- To evaluate web server functionality
- To measure system response time
- To assess operational stability under continuous usage

8.2 Unit Testing

Each component was tested individually before full system integration.

8.2.1 Sensor Testing

- DHT22 sensor readings were compared against standard thermometer and hygrometer values.
- PIR sensor output was tested by introducing controlled motion in front of the sensor.

- Multiple readings were recorded to verify consistency.

Result:
Sensor readings showed stable output with minor acceptable variations.

8.2.2 Relay Testing

- The relay module was triggered manually using test code.
- Switching behavior was observed under different conditions.
- Load switching was tested with safe low-voltage devices.

Result:
Relay activation and deactivation occurred correctly without delay.

8.2.3 LCD Testing

- Display outputs were verified for clarity and correct formatting.
- Continuous updates were tested for flickering or refresh errors.

Result:
Display functioned correctly with stable real-time updates.

8.3 Test Cases

8.3.1Integration Testing

After successful unit testing, complete system integration was performed. The

following scenarios were tested:

Test Case	Input Condition	Expected Output	Result
Motion detected	**PIR = HIGH**	**Relay ON**	**Passed**
No motion	**PIR = LOW**	**Relay OFF**	**Passed**

Temperature above threshold executed	**DHT value > set limit Passed**	**Control logic**
Web page accessed	**Client connected Passed**	**Real-time data displayed**
Power restart	**System reboot Passed**	**Auto initialization**

All integration test cases were successfully executed.

8.3.2 Performance Testing

Performance evaluation was conducted based on:

1. Response Time

Measured time between:

Sensor detection → Relay activation Average

response time observed:
Less than 1 second, confirming low-latency edge processing.

2. Network Performance

Web page loading time tested on connected devices. Data

refresh delay measured.

Observed result:
Real-time updates with negligible delay within local network.

3. Continuous Operation Test

The system was operated continuously for extended duration to evaluate stability. No

unexpected resets observed

No overheating detected

No memory crashes occurred

8.3.3 Reliability Testing

System reliability was tested by:

Disconnecting and reconnecting power supply Introducing

network interruption

Running the system for long-duration cycles

The system automatically reinitialized and resumed operation without manual intervention.

8.3.4 Security Testing

Basic security validation included:

- Access point password authentication
- Restricted web interface access
- Verification against unauthorized device connection The

system maintained secure local communication.

8.3.5 Digital Twin Synchronization Testing

To validate real-time synchronization:

- Sensor values were monitored simultaneously on:

    1. LCD display
    2. Web interface
    3. Internal processing logic

All displayed values matched physical sensor inputs accurately, confirming proper synchronization between physical and virtual system states.

8.3.6 Limitations Observed During Testing

PIR sensor sensitivity affected by environmental noise DHT22

slight delay in rapid temperature changes

Web interface refresh rate dependent on browser reload

These limitations are inherent to component specifications and not architectural faults.

9. SCREENS

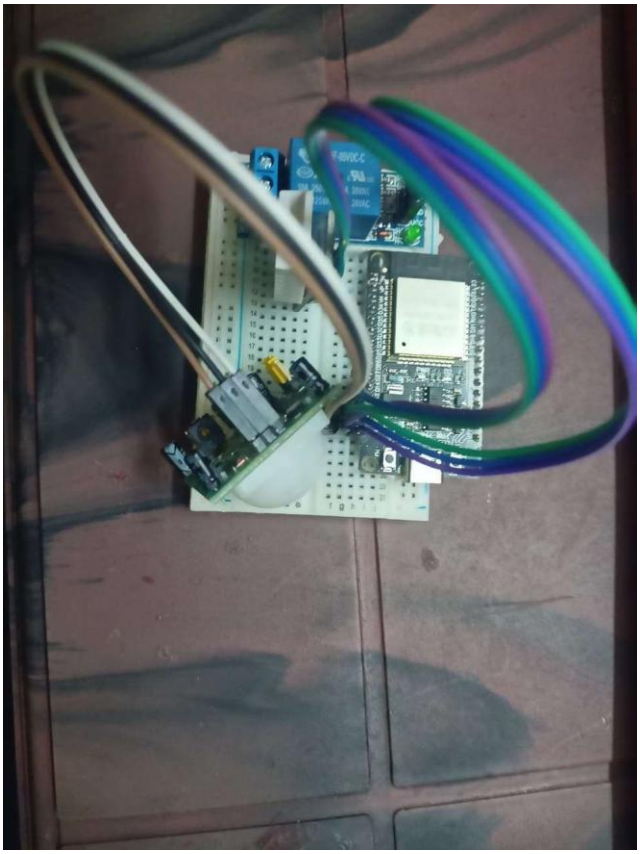

Fig 9.1: Implementation SS 1

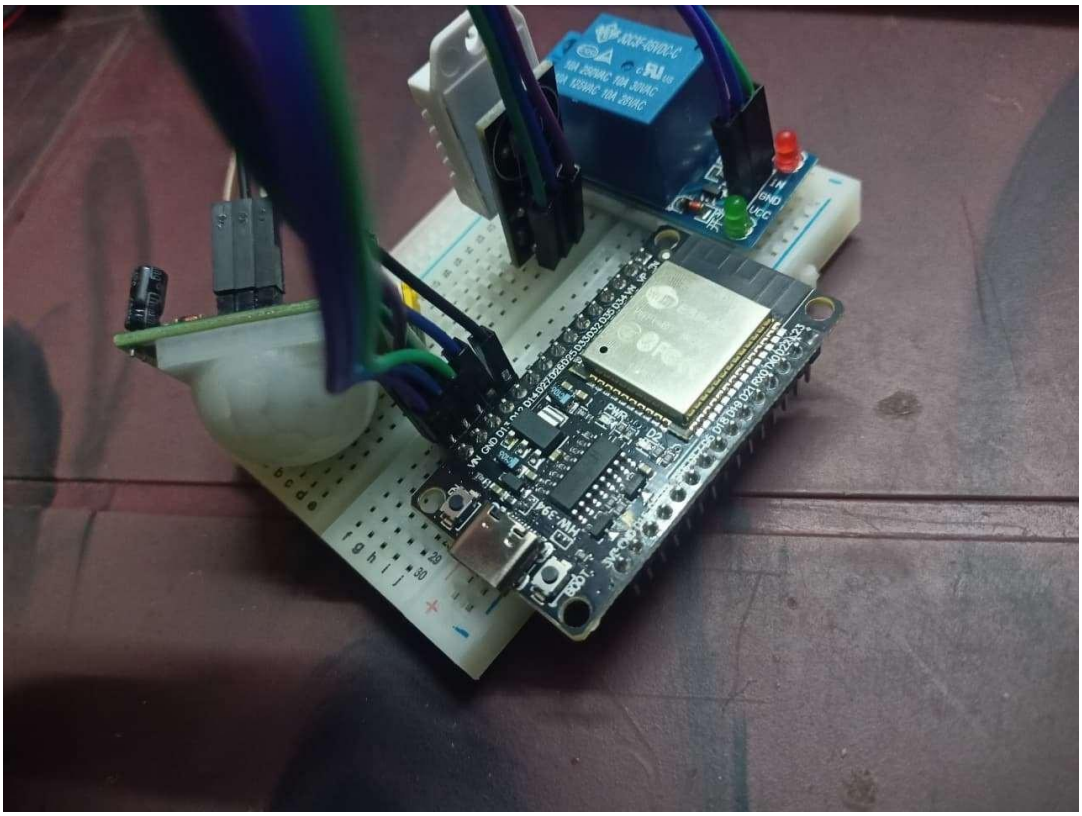

Fig 9.2: Implementation SS 2

10.CONCLUSION

The proposed Edge-Integrated Digital Twin IoT System successfully demonstrates the implementation of a real-time, low-latency, and energy-efficient embedded architecture. By shifting data processing from centralized cloud systems to localized edge devices, the system significantly reduces communication delay, bandwidth usage, and dependency on continuous internet connectivity.

The integration of digital twin modeling with edge computing enhances real-time synchronization between the physical system and its virtual representation. This enables predictive monitoring, intelligent control decisions, and improved operational reliability. The modular architecture ensures scalability, allowing additional sensors, communication layers, or AI-driven decision mechanisms to be incorporated in future enhancements.

Through systematic implementation and testing, the system validated:

- Accurate real-time sensor data acquisition
- Reliable actuator control through edge-based decision logic
- Stable embedded web server hosting
- Low response time operation
- Consistent digital twin synchronization

The project proves that hybrid edge-digital twin architectures are practical and efficient for modern IoT and Industry 4.0 applications. It establishes a strong foundation for developing scalable, autonomous, and intelligent embedded systems suitable for smart environments, industrial automation, and real-time monitoring applications.

11. REFERENCES

Below references are formatted in IEEE style.

[1] J. Chen, H. Yu, C. Lin, and R. Zhang, “Edge Intelligence Meets Digital Twins: A Smart Grid Perspective,” *IEEE Access*, 2022.

[2] M. A. Qadir, M. Naeem, and W. Ejaz, “Digital Twin-Assisted Multi-Layer Networks for Energy-Efficient IoT Communications,” *Computer Communications*, 2025.

[3] “Integration of Industrial IoT and Digital Twin for Intelligent Control,” *Machines Journal*, vol. 13, no. 10, 2025.

[4] S. K. Routray, “Cloud and Edge Computing for Industry 4.0,” *IEEE Computer Society*, 2025.

[5] GitHub Repository:

https://github.com/harishnethi/Batch_18

[6] Live Project Demo:

http://192.168.4.1/